

PROYECTO FINAL

Análisis Numérico de la Evolución del Precio de Bitcoin (BTC/USDT)



ASIGNATURA:
MÉTODOS NUMÉRICOS

INTEGRANTES:

ERICK TOAPANTA
HEIDY TENELEMA
NICOLÁS YUGSI
SEBASTIAN BRAVO

FECHA DE ENTREGA:
05 DE FEBRERO DE 2026

Contenido

INTRODUCCIÓN.....	1
OBJETIVOS.....	1
MARCO TEÓRICO	1
PRERREQUISITOS	3
DESARROLLO.....	4
CONCLUSIONES	14
RECOMENDACIONES.....	14
REFERENCIAS.....	15

INTRODUCCIÓN

El proyecto seleccionado lleva por título *Análisis Numérico de la Evolución del Precio de Bitcoin (BTC/USDT)*. En términos sencillos este trabajo busca construir una herramienta que permita entender el movimiento real del precio de esta moneda digital evitando quedarse solo con la gráfica visual que todos ven. La propuesta se basa en tomar los datos históricos obtenidos de un archivo real y aplicarles métodos numéricos para transformar esos puntos dispersos en una curva continua. Es similar a dibujar una trayectoria perfecta que pase por todos los datos usando fórmulas en lugar de una regla.

Respecto al problema de la vida real que se aborda es la incertidumbre y la volatilidad que presentan los activos digitales y más cuando dependen de decisiones subjetivas. En la actualidad muchas personas invierten en criptomonedas basándose solo en la intuición lo cual tiene un riesgo alto debido a los cambios bruscos de precio. Al aplicar ingeniería es posible calcular variables que no se ven a simple vista como la velocidad o la aceleración del precio lo que permite tomar decisiones basadas en cálculos y reducir el riesgo de pérdidas. Básicamente se usa la matemática para intentar ponerle orden al comportamiento del mercado.

OBJETIVOS

- Desarrollar un sistema computacional para procesar datos históricos de Bitcoin y generar modelos de tendencia mediante métodos numéricos.
- Evaluar el comportamiento del mercado aplicando regresión polinomial e interpolación por tramos, con el fin de proyectar trayectorias de precio y verificar el cumplimiento de precios objetivos.
- Validar la exactitud de los resultados comparando las predicciones con los datos reales del mercado, usando métricas de error relativo para determinar si el modelo es útil en escenarios volátiles.

MARCO TEÓRICO

Fundamentos del Par BTC/USDT y Variables del Dataset

El análisis numérico de este proyecto se centra en el par comercial Bitcoin frente a Tether (BTC/USDT). Se seleccionó este par específico porque el Tether (USDT) actúa como una

moneda estable con paridad 1:1 respecto al dólar estadounidense. Esto permite que el modelo matemático aísle la volatilidad del Bitcoin sin introducir ruido proveniente de la inflación o devaluación de monedas. Para alimentar los algoritmos, se utiliza la estructura de datos estandarizada en finanzas conocida como OHLC (Open, High, Low, Close). Según Murphy [5], estas variables son la base para cualquier estudio técnico, ya que resumen la psicología del mercado en un intervalo de tiempo discreto. Las variables que conforman el dataset y que el sistema procesa son:

- **Time (Tiempo):** Marca temporal exacta del inicio de la vela (timestamp).
- **Open (Apertura):** Precio del activo al comenzar la hora.
- **High (Máximo):** El valor más alto alcanzado durante ese periodo.
- **Low (Mínimo):** El valor más bajo registrado en la hora.
- **Close (Cierre):** Precio final del activo al concluir la hora.
- **Volume (Volumen):** Cantidad total de dinero transaccionado en ese lapso.

Variables del Dataset Utilizadas

Aunque el archivo original contiene múltiples columnas para la construcción del modelo numérico se realizó una selección específica de variables que garantizan la continuidad necesaria para los algoritmos implementados:

1. **Variable Independiente (t):** Se derivó de la columna `open_time`.
2. **Variable Dependiente (y):** Se seleccionó la columna `close_price`.
3. **Variables Auxiliares:** Se conservaron `high_price` y `low_price` únicamente para la visualización de la volatilidad en la Fase 2.

Para garantizar que los cálculos del proyecto sean fiables, lo primero fue definir cómo la computadora maneja los números decimales. Se utilizó el estándar IEEE 754 de punto flotante en doble precisión, que es la norma en ingeniería para evitar que los datos se alteren. Según Chapra y Canale [1], trabajar con esta precisión es necesario cuando se realizan muchas operaciones seguidas, ya que evita que el error se acumule. Esto es

importante al procesar los precios del Bitcoin, que tienen muchos decimales y varían en milisegundos.

Una vez asegurada la calidad de los datos, el sistema funciona definiendo un intervalo de tiempo específico que actúa como conjunto de entrenamiento. Dentro de este periodo seleccionado, los precios reales suelen ser muy caóticos debido a la volatilidad del mercado. Para solucionar esto y estimar una tendencia, se aplicó la regresión por mínimos cuadrados. Burden y Faires [2] explican que este método permite trazar una curva que representa el comportamiento general del activo, minimizando el error global sin necesidad de coincidir con cada subida o bajada repentina. De esta forma, el algoritmo aprende la dirección del mercado basándose solo en datos históricos, eliminando la opinión subjetiva del inversor, tal como sugiere López de Prado [3] en el análisis financiero moderno.

Finalmente, para que el modelo no sea solo una línea rígida y pueda adaptarse a los cambios de precio con suavidad, se utilizó la interpolación por splines cúbicos. Nieves y Domínguez [4] señalan que esta técnica conecta los puntos de datos usando pequeños polinomios de tercer grado, lo que garantiza que la curva resultante sea continua y no tenga quiebres bruscos en las uniones. Gracias a esto, la trayectoria matemática del precio es fluida y derivable, permitiendo hacer estimaciones precisas sobre si el valor del activo alcanzará el objetivo propuesto dentro del rango de validación.

PRERREQUISITOS

Para la implementación y ejecución correcta de este proyecto es necesario cumplir con ciertos requerimientos técnicos y de información. En primer lugar se requiere un dominio intermedio del lenguaje de programación Python ya que esto es fundamental no solo para escribir los algoritmos de los métodos numéricos, sino para manejar librerías de cálculo vectorial como NumPy y manipulación de datos como Pandas. Además, para que el modelo sea útil y comprensible, se necesitan conocimientos en el desarrollo de interfaces gráficas, esto permite sacar los cálculos de la terminal y presentarlos en una aplicación web visual donde se puedan ver las curvas y modificar los parámetros en tiempo real.

El segundo pilar importante es la base de datos. El sistema no puede generar predicciones de la nada dado que se requiere un dataset histórico real del par BTC/USDT. Este archivo debe contener una estructura específica con las variables financieras estándar de cada

sesión como la fecha exacta, precio de apertura, precio más alto, precio más bajo y precio de cierre. Sin estos datos históricos organizados cronológicamente, el algoritmo no tiene insumos para realizar el entrenamiento de la regresión ni los puntos necesarios para trazar los splines cúbicos.

DESARROLLO

La construcción del modelo numérico se realizó siguiendo una metodología secuencial, donde la salida de una fase servía como dato de entrada para la siguiente. El proceso general consistió en transformar una base de datos de precios en datos fáciles de procesar y graficar. Todo el código fue escrito en Python utilizando las librerías NumPy y Pandas para la manipulación matricial, ya que, como menciona McKinney [6], estas herramientas permiten reducir el tiempo de cómputo en comparación con los bucles tradicionales.

A continuación, se detalla la lógica matemática y algorítmica aplicada en cada etapa del proyecto:

Fase 1: Preprocesamiento y Linealización

Lo primero que se abordó fue la calidad de los datos. El archivo original contenía precios con caracteres de formato (puntos de miles) que impedían su cálculo directo. Se implementó una función de limpieza para convertir estos textos al estándar IEEE 754, además como las fechas (timestamps) no son operables directamente en ecuaciones polinomiales, se realizó una linealización temporal. Esto significa que se transformó el tiempo real en una variable discreta t donde $t = 0$ es el inicio de la serie.

Ecuación de Linealización:

$$t_i = i ; i = [0, 1, 2, \dots, n - 1]$$

Donde n es el número total de registros horarios analizados.

```
*** REPORTE DE PROCESAMIENTO EXITOSO
Puntos de datos en ventana: 241
Rango temporal de t: [0, 240]
-----
      open_time  t  close_price
35 2017-08-20 00:00:00  0      4086.09
36 2017-08-20 01:00:00  1      4082.53
37 2017-08-20 02:00:00  2      4124.69
38 2017-08-20 03:00:00  3      4094.62
39 2017-08-20 04:00:00  4      4093.00
```

Fig. 1. Vista de los datos procesados tras la conversión a punto flotante y creación de variable t .

Funciones Implementadas:

- **cargar_y_limpiar_datos():** Es la función principal desarrollada en el script. Se encarga de leer el archivo CSV detectando automáticamente el separador (punto y coma).
- **normalizar_valor(texto):** Una rutina diseñada en este proyecto para corregir el formato de los precios. Elimina los puntos separadores de miles que venían en el dataset original para convertirlos en números de punto flotante válidos según el estándar IEEE 754.
- **pd.to_datetime():** Función de la librería Pandas utilizada para convertir la columna `open_time` en objetos de fecha manipulables, permitiendo posteriormente la creación de la variable matemática t .

Fase 2: Análisis de Volatilidad

Antes de ajustar cualquier curva, era necesario entender la volatilidad del mercado. Se graficó una banda de volatilidad utilizando los precios máximos (H_i) y mínimos (L_i) de cada hora. Matemáticamente, el precio de cierre (C_i) debe estar contenido siempre dentro de este intervalo para que el dato sea válido.

Desigualdad de Validación:

$$L_i \leq C_i \leq H_i$$

Esta visualización permitió confirmar que no existían datos atípicos que pudieran deformar los polinomios en las fases siguientes.

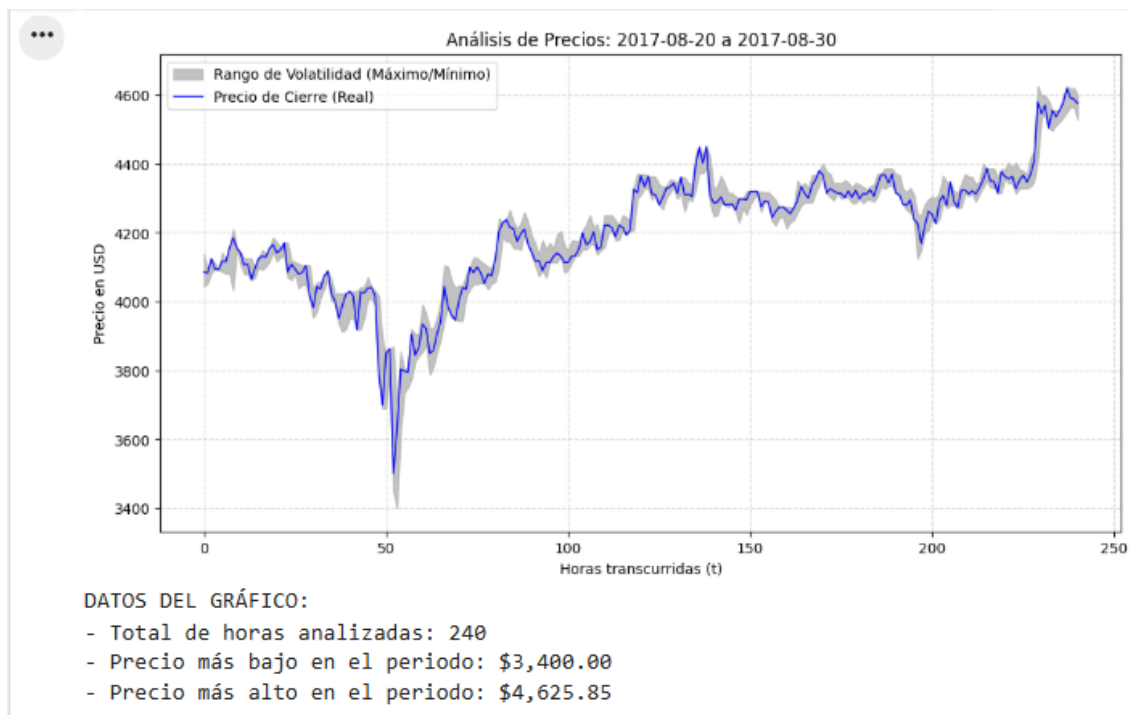


Fig. 2. Banda de volatilidad histórica del Bitcoin en el intervalo seleccionado.

Funciones Implementadas:

- **go.Figure():** Constructor principal de la librería Plotly utilizado para crear el lienzo gráfico interactivo.
- **go.Scatter(..., fill = 'tonexty'):** Se utilizó esta función para trazar las líneas de precios máximos (high_price) y mínimos (low_price). El parámetro fill='tonexty' es la clave lógica de esta fase, ya que rellena automáticamente el área entre ambas curvas para generar la sombra visual que representa la volatilidad del mercado.

Fase 3: Regresión por Mínimos Cuadrados

Para modelar la tendencia global, lo que se refiere hacia donde se dirige la gráfica desde una perspectiva general, se utilizó el método de Mínimos Cuadrados. El objetivo fue encontrar los coeficientes de un polinomio $P(t)$ de grado n que minimizara el error cuadrático total respecto a los datos reales.

Modelo Polinomial:

$$P(t) = a_n t^n + a_{n-1} t^{n-1} + \dots + a_1 t + a_0$$

El algoritmo resuelve un sistema de ecuaciones lineales para hallar los coeficientes a_k tal que la suma de los residuos al cuadrado sea mínima. En el proyecto se optó por un polinomio de hasta tercer grado porque ofrece flexibilidad para detectar cambios de tendencia sin tener un sobreajuste.

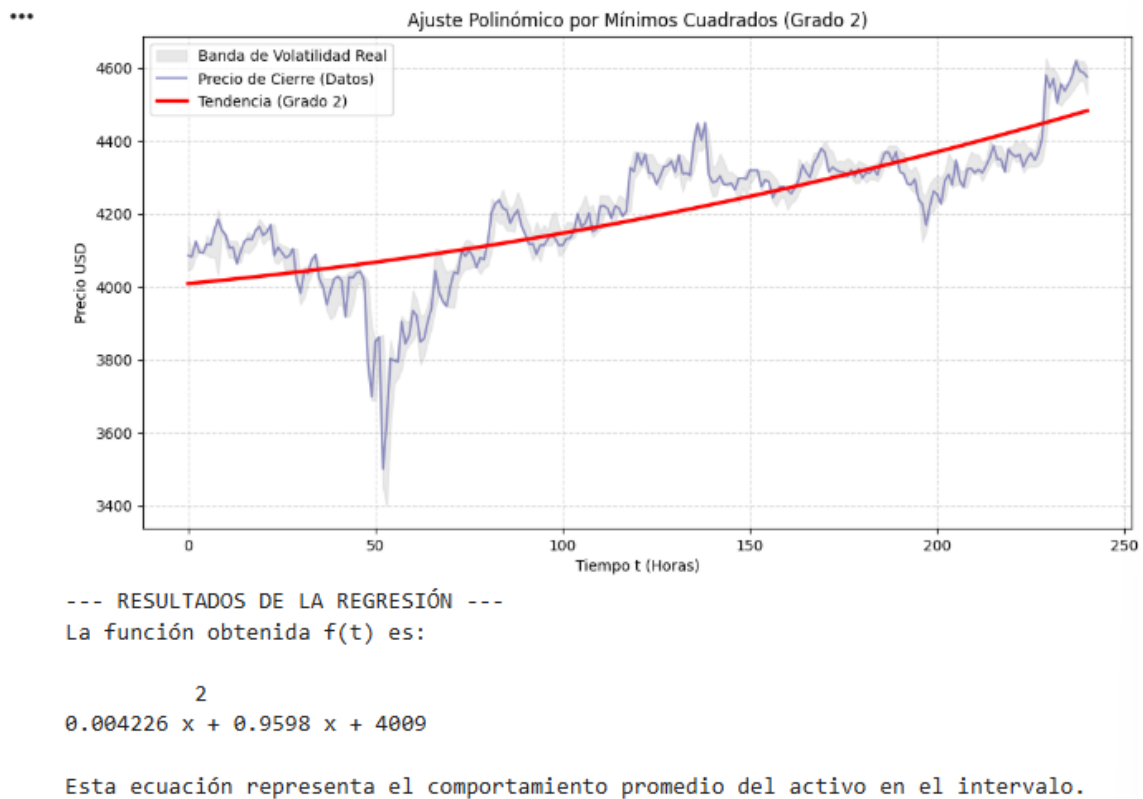


Fig. 3. Ajuste de tendencia polinomial sobre los datos reales.

Funciones Implementadas:

- **np.polyfit(x, y, grado):** Función de la librería NumPy que aplica el método de Mínimos Cuadrados. Recibe el vector de tiempo t y los precios de cierre, devolviendo los coeficientes del polinomio que minimizan el error cuadrático global.
- **np.poly1d(coef):** Toma los coeficientes calculados por la función anterior y construye un objeto matemático funcional. Esto permite evaluar el polinomio $P(t)$ en cualquier punto del tiempo para dibujar la línea de tendencia roja.
- **r2_score():** Función de la librería Scikit-learn utilizada para calcular el coeficiente de determinación, métrica estadística que nos indica qué tan bien se ajusta la tendencia a los datos reales.

Fase 4: Interpolación mediante Splines Cúbicos

Mientras que la regresión nos da la tendencia general, los Splines Cúbicos se usaron para conectar los puntos de forma suave. A diferencia del polinomio único, aquí se construyeron múltiples polinomios $S_j(t)$. La condición matemática clave que se programó es que la función, su primera derivada (velocidad del precio) y su segunda derivada (aceleración) sean iguales en los puntos de unión.

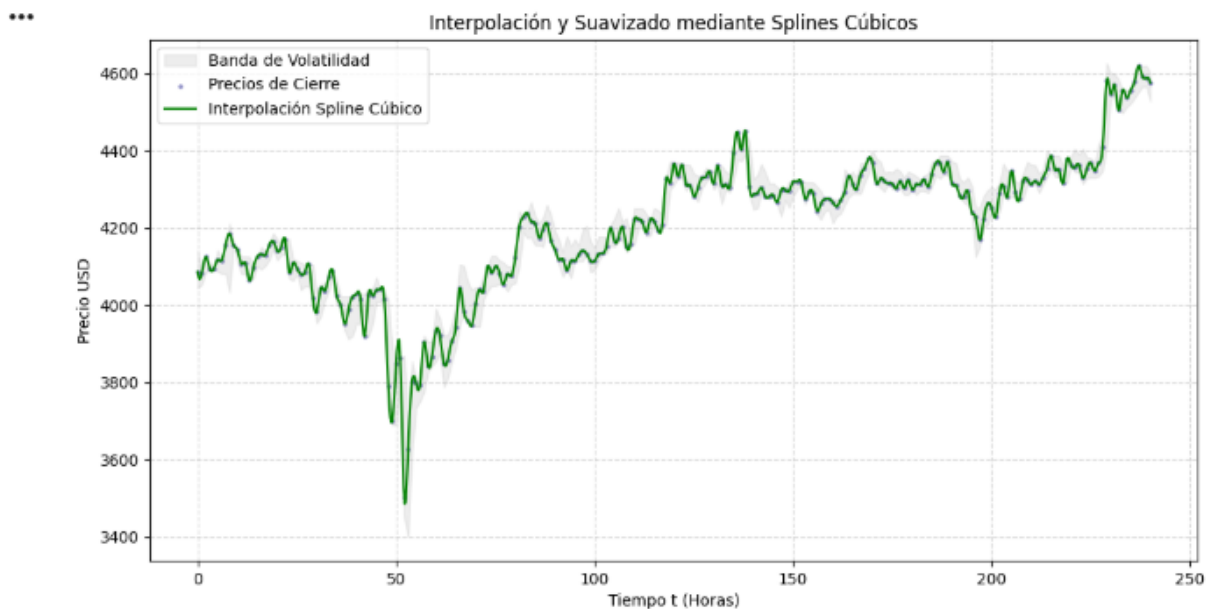
Condición de Continuidad:

$$S_j(t_{j+1}) = S_{j+1}(t_{j+1}),$$

$$S'_j(t_{j+1}) = S'_{j+1}(t_{j+1}),$$

$$S''_j(t_{j+1}) = S''_{j+1}(t_{j+1})$$

Esto garantiza que la curva no tenga picos bruscos, simulando mejor la fluidez de un mercado financiero real.



Se ha generado una función $S(t)$ continua y derivable en todo el intervalo.
Número de nodos de interpolación: 241

Fig. 4. Interpolación suave de la trayectoria del precio mediante splines.

Funciones Implementadas:

- **CubicSpline(t, y, bc_type='natural')**: Función de la librería `scipy.interpolate`. Es el núcleo del proyecto ya que genera la estructura matemática que conecta todos los puntos de precio mediante polinomios cúbicos por tramos, garantizando suavidad en las uniones.
- **spline(t, 1)**: Método derivado del objeto `Spline`. Se programó para calcular numéricamente la primera derivada ($S'(t)$), que representa la velocidad de cambio del precio en USD/hora.
- **spline(t, 2)**: Método para calcular la segunda derivada ($S''(t)$), utilizada para graficar la aceleración del mercado y detectar cambios de fuerza en la tendencia.

Fase 5: Estimación y Validación del Modelo

Finalmente, se puso a prueba el modelo. Para validar si las predicciones eran confiables, se aplicó una técnica de Hold-out, lo que se refiere a ocultar las últimas 48 horas de datos reales al algoritmo y se le pidió que proyectara el precio para ese lapso.

Se comparó la proyección (y_{pred}) con la realidad oculta (y_{real}) calculando el Error Cuadrático Medio (RMSE) y el Error Relativo Porcentual.

Fórmulas de Error:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_{real,i} - y_{pred,i})^2}$$

$$Error\ Relativo\ (\%) = \frac{RMSE}{\bar{y}_{real}} * 100$$

Si el error relativo resultaba menor al 10% y la dirección de la curva coincidía con el precio objetivo del usuario, el sistema entrega una recomendación positiva caso contrario entrega una advertencia de pérdida.

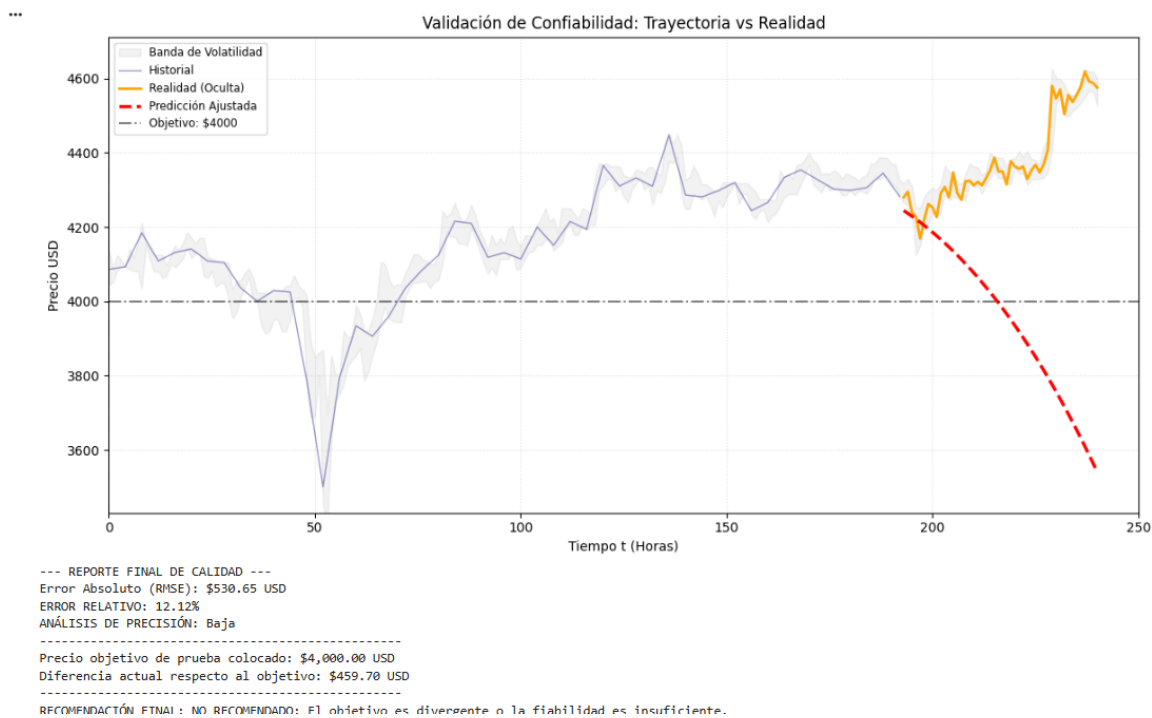


Fig. 5. Validación del modelo frente a datos reales ocultos y métricas de error resultantes.

Funciones Implementadas:

- **df.iloc[:n]:** Se utilizó la técnica de *slicing* (recorte) de Pandas para separar manualmente los datos en dos conjuntos: entrenamiento (historial) y prueba (las últimas horas ocultas).
- **mean_squared_error():** Función de Scikit-learn que compara vectorialmente los precios reales ocultos con los proyectados por el modelo, devolviendo el Error Cuadrático Medio (RMSE).
- **np.linspace() y timedelta():** Utilizadas en conjunto para generar el vector de tiempo futuro sobre el cual se proyecta la curva del modelo para visualizar la predicción más allá de los datos conocidos.

Análisis de Resultados

Tras ejecutar el modelo con los datos históricos, el sistema genera un reporte final que permite evaluar la precisión de la trayectoria calculada frente al comportamiento real del mercado. En esta fase de validación, se observa cómo la curva de interpolación se ajusta a

los precios de cierre, mientras que la proyección a futuro intenta alcanzar el objetivo definido por el usuario.

El indicador clave para determinar el éxito del experimento es el Error Cuadrático Medio (RMSE). En los resultados, se visualiza si la desviación del modelo se mantiene dentro de los márgenes aceptables, esto quiere decir que un error relativo bajo (inferior al 15%) indica que el sistema logró capturar la inercia del precio de manera efectiva. Además, el análisis de resultados muestra una comparativa visual entre el precio actual y el precio objetivo, emitiendo una recomendación. Si la pendiente de la curva proyectada es coherente con la meta y el error de cálculo es mínimo, el sistema valida la operación como recomendada, demostrando que las herramientas numéricas aplicadas son capaces de tomar decisiones al recibir un periodo de entrenamiento adecuado.

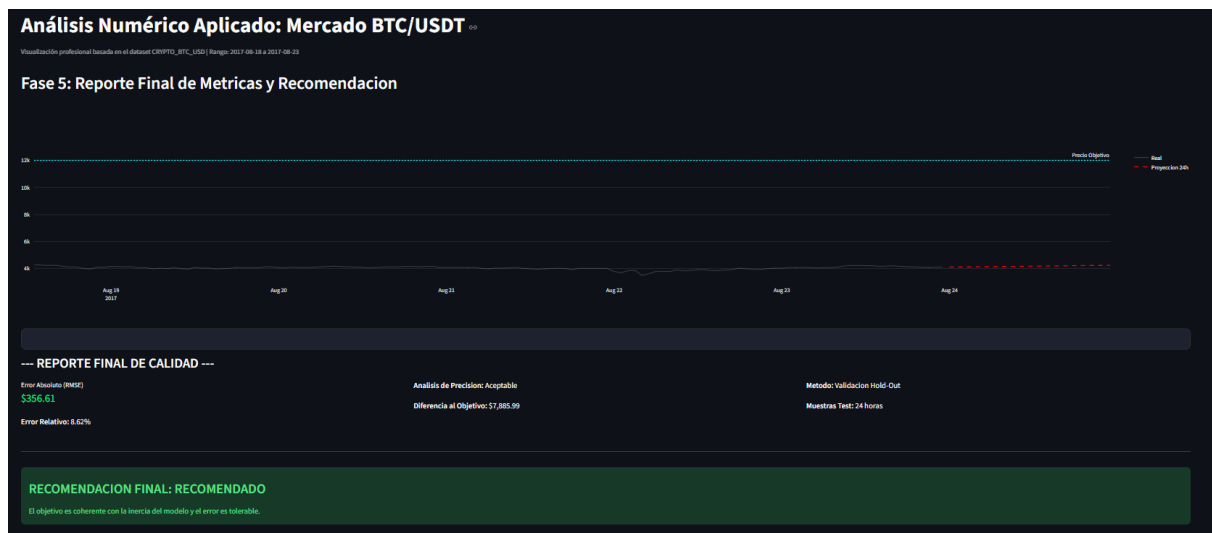


Fig. 6. Reporte final de métricas, cálculo de error y validación de la proyección frente al precio objetivo.

Flujograma de funciones implementadas

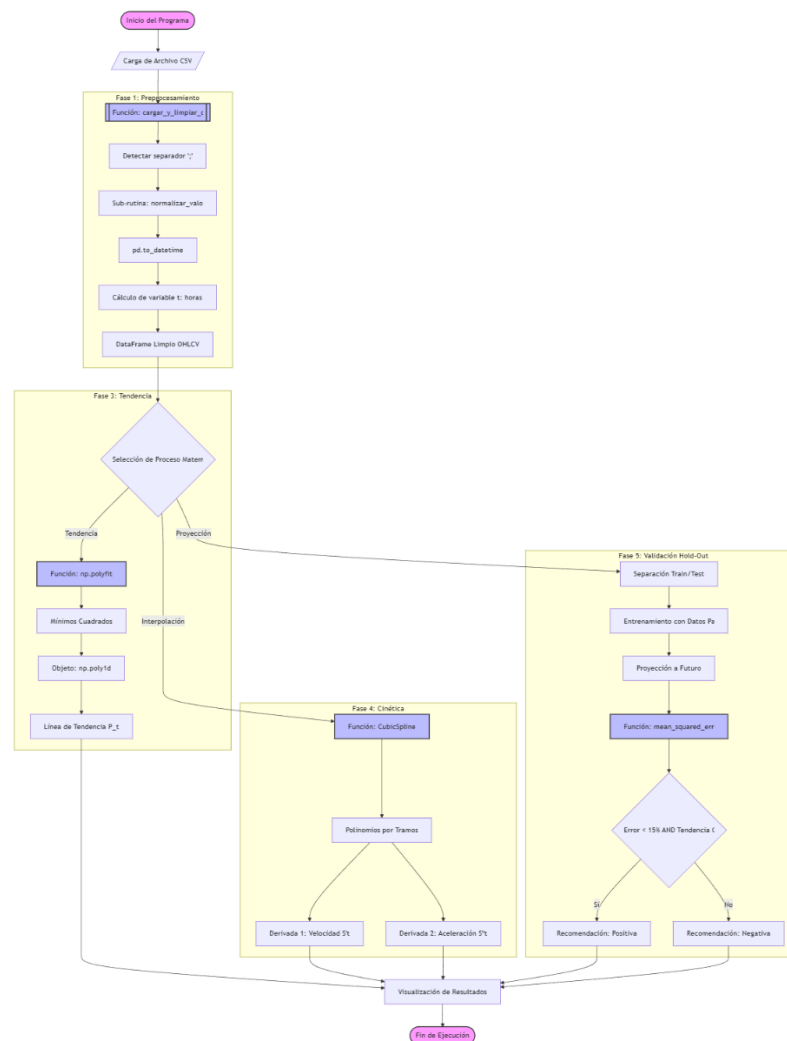


Fig. 7. Diagrama de flujo funcional de la lógica de ejecución y funciones implementadas.

Enlace del Repositorio y Estadísticas de Contribución

Para facilitar el acceso y la revisión del proyecto, se ha subido todo el código a la plataforma GitHub. En el siguiente enlace se encuentra disponible tanto el programa principal como los archivos de prueba utilizados:

<https://github.com/Nico222614/Dominio-Grupo-4-Finanzas>

Estadísticas de Contribución

El desarrollo del software fue un esfuerzo conjunto. Las siguientes gráficas tomadas de la plataforma muestran la actividad de cada integrante del grupo, evidenciando el trabajo colaborativo realizado para completar las distintas fases del proyecto.

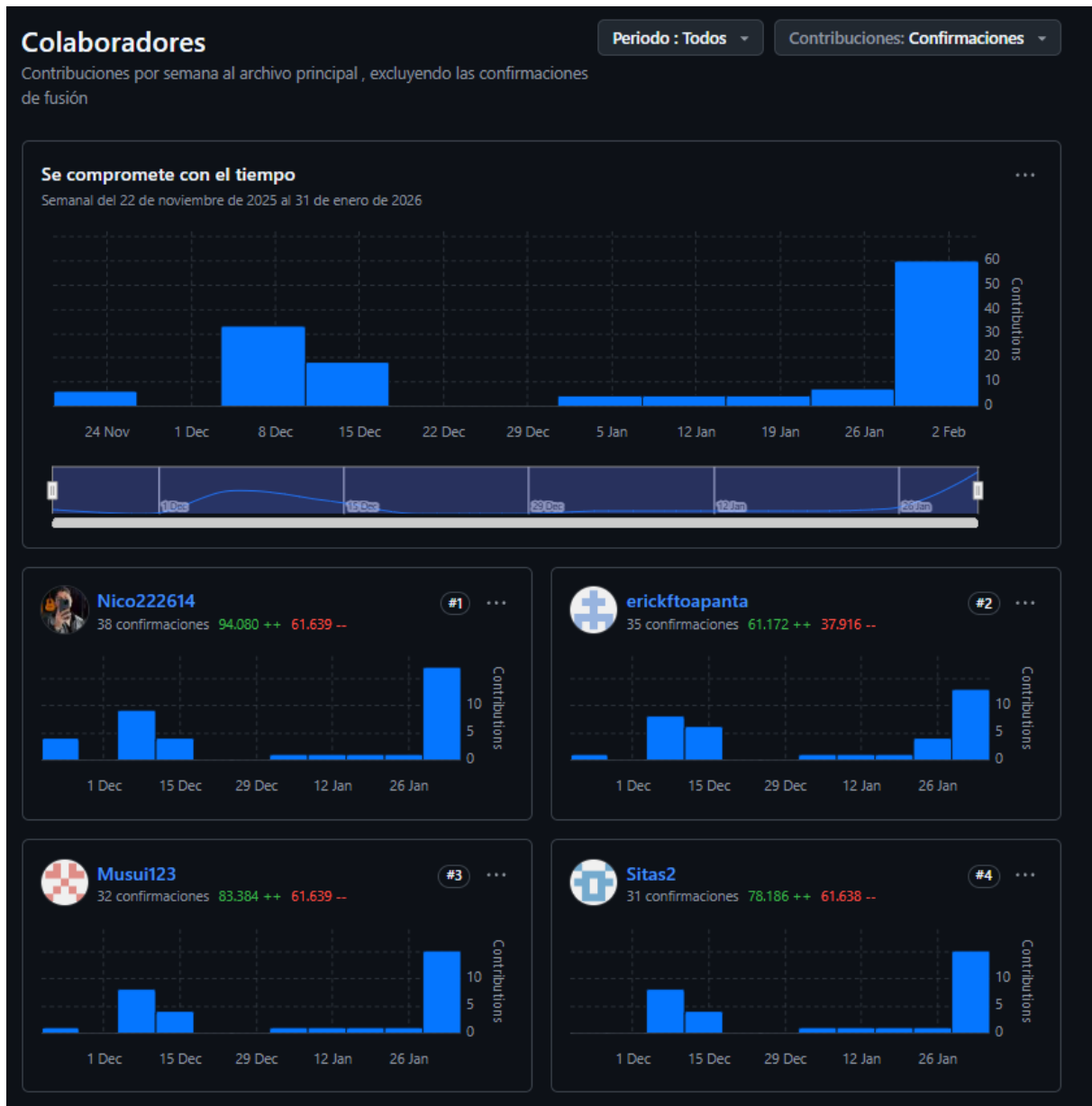


Fig. 8. Registro de actividad y participación del equipo en el proyecto.

CONCLUSIONES

Al finalizar el desarrollo del proyecto se pudo confirmar que los resultados obtenidos son positivos siempre que se trabaje en rangos de tiempo cortos. Se observó que el sistema logra representar el movimiento real de los datos y permite tomar una decisión clara basándose en si el margen de error se mantiene bajo. Sin embargo se hizo evidente que en momentos de cambios muy bruscos la herramienta pierde precisión lo que demuestra que el modelo funciona como un apoyo técnico pero no puede predecir eventos totalmente inesperados del mercado financiero. En general el trabajo realizado permitió comprobar que es posible poner orden a datos que parecen caóticos y obtener una respuesta lógica sobre el comportamiento de un activo en el futuro inmediato.

RECOMENDACIONES

Es importante considerar que este sistema es un modelo básico que resuelve el problema planteado pero funciona principalmente en un entorno controlado. Para obtener resultados que sean realmente óptimos es fundamental tener mucho cuidado al definir el periodo de entrenamiento y el rango de datos que se le entrega al programa al inicio. Si el rango es muy extenso o demasiado pequeño el cálculo final puede verse afectado por lo que se sugiere ajustar estas variables según que tan estable esté el precio en ese momento. También se aconseja no usar el modelo para metas a muy largo plazo sino enfocarse en proyecciones rápidas donde la fiabilidad de la respuesta es mucho mayor.

REFERENCIAS

- [1] S. C. Chapra y R. P. Canale, *Métodos numéricos para ingenieros*, 7ma ed. México: McGraw-Hill, 2015.
- [2] R. L. Burden y J. D. Faires, *Análisis numérico*, 9na ed. México: Cengage Learning, 2011.
- [3] M. López de Prado, *Advances in Financial Machine Learning*. Nueva Jersey: John Wiley & Sons, 2018.
- [4] A. Nieves y F. Domínguez, *Métodos numéricos aplicados a la ingeniería*, 4ta ed. México: Grupo Editorial Patria, 2014.
- [5] J. J. Murphy, *Technical Analysis of the Financial Markets: A Comprehensive Guide to Trading Methods and Applications*. Nueva York: New York Institute of Finance, 1999.
- [6] W. McKinney, *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*, 2da ed. Sebastopol: O'Reilly Media, 2017.